Übungsblatt 6 – Lösungen

Optimierung

Teil A: Auf Papier

Aufgabe A1 (Taylor in zwei Variablen)

Das Taylorpolynom k-ter Ordnung von f in p ist $T_k(h) = \sum_{j=0}^{k} \frac{1}{j!} (h \cdot \nabla)^j f(p)$, bis zur zweiten Ordnung also $T_2(h) = f(p) + \nabla f(p) \cdot h + \frac{1}{2}\, h^\top H_f(p)\, h$.

(a) Berechnen Sie T_2 für $f(x, y) = e^x \cos y$ in $p = (0, 0)$.

(b) Vergleichen Sie $T_2(0{,}1;\ 0{,}2)$ mit $f(0{,}1;\ 0{,}2)$, und berechnen Sie das Glied dritter Ordnung. Wie groß ist der Fehler von T_3?

(c) Sei $g(x, y) = x^2 y + 3y^2$ und $p = (1, 2)$. Berechnen Sie T_2 bei $h = (\frac{1}{10}, -\frac{1}{10})$ und vergleichen Sie mit $g(1{,}1;\ 1{,}9) = 13{,}129$. Warum ist T_3 exakt?

Hinweis: Zu (b): Es ist $f(0{,}1;\ 0{,}2) = 1{,}083141\ldots$, und in $(h \cdot \nabla)^3 f$ stehen die dritten Ableitungen: $f_{xxx}\, h_1^3 + 3 f_{xxy}\, h_1^2 h_2 + 3 f_{xyy}\, h_1 h_2^2 + f_{yyy}\, h_2^3$.

Lösung. **(a)** Nach x abgeleitet bleibt $e^x \cos y$ stehen, nach y wechseln cos und sin. In $(0, 0)$: $f = 1$, $f_x = 1$, $f_y = -e^x \sin y = 0$, $f_{xx} = 1$, $f_{xy} = 0$, $f_{yy} = -e^x \cos y = -1$. Also $\nabla f(0) = (1, 0)$, $H_f(0) = \left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$ und

$$T_2(h) = 1 + h_1 + \tfrac{1}{2}\,(h_1^2 - h_2^2).$$

(b) $T_2(0{,}1;\ 0{,}2) = 1 + 0{,}1 + \frac{1}{2}\,(0{,}01 - 0{,}04) = 1{,}085$; der Fehler gegen $f = 1{,}083141$ ist $-0{,}00186$. Die dritten Ableitungen in 0 sind $f_{xxx} = 1$, $f_{xxy} = 0$, $f_{xyy} = -1$, $f_{yyy} = 0$, das Glied dritter Ordnung also $\frac{1}{6}\,(h_1^3 - 3h_1 h_2^2) = \frac{1}{6}\,(0{,}001 - 0{,}012) = -0{,}001833$. Damit ist $T_3 = 1{,}083167$ und der Fehler nur noch $-2{,}6 \cdot 10^{-5}$ – etwa so groß wie das Glied vierter Ordnung, $\frac{1}{24}\,(h_1^4 - 6h_1^2 h_2^2 + h_2^4) = -2{,}9 \cdot 10^{-5}$.

(c) $g(1, 2) = 2 + 12 = 14$, $\nabla g = (2xy,\ x^2 + 6y) = (4,\ 13)$ und $H_g = \left(\begin{smallmatrix} 2y & 2x \\ 2x & 6 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 4 & 2 \\ 2 & 6 \end{smallmatrix}\right)$. Mit $h = (0{,}1;\ -0{,}1)$ ist der lineare Teil $0{,}4 - 1{,}3 = -0{,}9$ und der quadratische $\frac{1}{2}\,(4 \cdot 0{,}01 + 2 \cdot 2 \cdot 0{,}1 \cdot (-0{,}1) + 6 \cdot 0{,}01) = 0{,}03$, also $T_2 = 13{,}13$ und der Fehler $-0{,}001$. Von den dritten Ableitungen ist nur $g_{xxy} = 2$ nicht null; das Glied dritter Ordnung ist $\frac{1}{6} \cdot 3\, g_{xxy}\, h_1^2 h_2 = h_1^2 h_2 = -0{,}001$, und $T_3 = 13{,}129$ ist exakt. Denn g ist ein Polynom vom Grad 3: Alle vierten Ableitungen sind null, das Restglied verschwindet.

Aufgabe A2 (Hoch-, Tief- und Sattelpunkte)

Sei

$$f(x, y) = x^3 - 3x + y^3 - 3y.$$

(a) Berechnen Sie ∇f, alle kritischen Punkte und in jedem die Hesse-Matrix. Entscheiden Sie mit dem Hesse-Kriterium, wo ein Tiefpunkt, wo ein Hochpunkt und wo ein Sattelpunkt liegt.

(b) Bestätigen Sie den Tiefpunkt ohne Hesse-Kriterium: Zeigen Sie $f(1+h,1+k)-f(1,1) = h^2(3+h)+k^2(3+k)$. Auf welcher Menge ist damit $f \geq f(1,1)$? Wie folgt daraus der Hochpunkt?

(c) Bestätigen Sie den Sattelpunkt $(1,-1)$ mit den Schnittkurven $t \mapsto f(1+t,-1)$ und $t \mapsto f(1,-1+t)$. Warum ist auch $(-1,1)$ ein Sattelpunkt?

(d) Hat f ein globales Minimum oder Maximum?

(e) Für $q(x,y) = x^2+3xy+y^2$ haben beide Schnittkurven längs der Achsen, $q(t,0) = t^2$ und $q(0,t) = t^2$, in $t=0$ ein Minimum. Was sagt das Kriterium in zwei Dimensionen über $(0,0)$? Berechnen Sie $q(t,-t)$ und die Eigenwerte der Hesse-Matrix von q.

Hinweis: Zu (b): $f(-x,-y) = -f(x,y)$. Zu (c): $f(x,y) = f(y,x)$. Zu (e): Die Eigenvektoren sind $(1,1)$ und $(1,-1)$.

Lösung. **(a)** $\nabla f(x,y) = (3x^2-3,\ 3y^2-3)$ ist null genau für $x^2=1$ und $y^2=1$. Die kritischen Punkte sind $(1,1)$, $(-1,-1)$, $(1,-1)$ und $(-1,1)$. Die gemischten Ableitungen sind 0, also

$$H_f(x,y) = \begin{pmatrix} 6x & 0 \\ 0 & 6y \end{pmatrix}, \qquad \det H_f = 36\,xy.$$

In $(1,1)$ ist $\det H_f = 36 > 0$ und $a = 6 > 0$: ein Tiefpunkt mit $f(1,1) = -4$. In $(-1,-1)$ ist $\det H_f = 36 > 0$ und $a = -6 < 0$: ein Hochpunkt mit $f(-1,-1) = 4$. In $(1,-1)$ und $(-1,1)$ ist $\det H_f = -36 < 0$: Sattelpunkte. In Eigenwerten: $6x$ und $6y$, also $6, 6$ im Tiefpunkt, $-6,-6$ im Hochpunkt und $6,-6$ in den Sattelpunkten.

(b) Es ist $(1+h)^3 - 3(1+h) = -2+3h^2+h^3$, ebenso für k, und $f(1,1) = -4$. Die Differenz ist also $3h^2+h^3+3k^2+k^3 = h^2(3+h)+k^2(3+k)$, und für $h,k \geq -3$ ist sie ≥ 0: Es gilt $f(x,y) \geq -4 = f(1,1)$ für alle $x,y \geq -2$. Diese Menge enthält die Kugel vom Radius 3 um $(1,1)$. Wegen $f(-x,-y) = -f(x,y)$ ist $f(x,y) \leq 4 = f(-1,-1)$ für $x,y \leq 2$: das lokale Maximum. Die Rechnung zeigt auch, woher das Kriterium kommt: Der quadratische Teil $3h^2+3k^2$ ist $\frac{1}{2}(h,k)\,H_f(1,1)\binom{h}{k}$, und der kubische Rest h^3+k^3 ist klein gegen h^2+k^2.

(c) Es ist $f(1,-1) = -2+2 = 0$ und

$$f(1+t,-1) = t^2(3+t), \qquad f(1,-1+t) = t^2(t-3).$$

Für $0 < |t| < 3$ ist die erste Zahl > 0 und die zweite < 0. Jede Kugel um $(1,-1)$ enthält also Punkte mit $f > f(1,-1)$ und Punkte mit $f < f(1,-1)$: kein lokales Minimum, kein lokales Maximum. In x-Richtung liegt ein Minimum, in y-Richtung ein Maximum – die Eigenrichtungen zu den Eigenwerten 6 und -6. Wegen $f(x,y) = f(y,x)$ gilt dasselbe mit vertauschten Rollen in $(-1,1)$.

(d) Nein: $f(x,0) = x^3-3x$ wächst für $x \to \infty$ über jede Schranke und fällt für $x \to -\infty$ unter jede; schon $f(-3,0) = -18 < -4$ und $f(3,0) = 18 > 4$. Das Hesse-Kriterium ist eine lokale Aussage.

(e) $\nabla q = (2x+3y,\ 3x+2y)$ ist nur in $(0,0)$ null, und $H_q = \left(\begin{smallmatrix} 2 & 3 \\ 3 & 2 \end{smallmatrix}\right)$ hat $\det H_q = 4-9 = -5 < 0$: Nach dem Kriterium ist $(0,0)$ ein Sattelpunkt, obwohl beide Schnittkurven längs der Achsen dort ein Minimum haben. Die Eigenwerte sind 5 zu $(1,1)$ und -1 zu $(1,-1)$, und tatsächlich ist $q(t,-t) = t^2-3t^2+t^2 = -t^2 < 0 = q(0,0)$ für $t \neq 0$. Längs der Achsen sieht man nur die Diagonaleinträge der Hesse-Matrix; hier entscheidet der gemischte Term $3xy$. Es ist $q(v) = \frac{1}{2}\,v^\top H_q\,v$ exakt, in Richtung $(1,-1)$ also $q(t,-t) = \frac{1}{2}\cdot(-1)\cdot\|(t,-t)\|^2 = -t^2$.

Aufgabe A3 (Gradientenabstieg)

Der Gradientenabstieg mit Schrittweite $t > 0$ ist $x_{k+1} = x_k - t\,\nabla f(x_k)$.

(a) Sei $f(x) = (x-2)^2$, $t = \frac{1}{4}$ und $x_0 = 6$. Berechnen Sie x_1, x_2, x_3 und zeigen Sie $x_{k+1} - 2 = \frac{1}{2}(x_k - 2)$. Wie viele Schritte braucht man, bis $|x_n - 2| \le \frac{1}{1000}$ ist?

(b) Für welche Schrittweiten t konvergiert das Verfahren in (a) für jeden Startwert? Was passiert bei $t = 1$ und bei $t = \frac{6}{5}$ mit $x_0 = 6$?

(c) Sei nun $f(x, y) = \frac{1}{2}(x^2 + 4y^2)$ mit Minimum in $(0, 0)$. Berechnen Sie ∇f, dann zwei Schritte mit $t = \frac{1}{5}$ von $(x_0, y_0) = (4, 1)$ aus, und geben Sie (x_n, y_n) für alle n an.

(d) Für welche t konvergiert das Verfahren in (c) von jedem Startwert aus gegen $(0, 0)$? Was passiert bei $t = \frac{1}{2}$?

(e) Sei $\rho(t) = \max\{\,|1-t|,\ |1-4t|\,\}$; dann ist $\|(x_n, y_n)\| \le \rho(t)^n \,\|(x_0, y_0)\|$. Für welches t ist $\rho(t)$ am kleinsten? Wie viele Schritte garantiert die Schranke bis $\|(x_n, y_n)\| \le \frac{1}{1000}\,\|(x_0, y_0)\|$ mit diesem t, wie viele mit $t = \frac{1}{5}$?

Hinweis: In jedem Schritt wird der Fehler (bzw. jede Koordinate) mit einem festen Faktor multipliziert. Zu (e): $|1-t|$ und $|1-4t|$ über t skizzieren.

Lösung. **(a)** Mit $f'(x) = 2(x-2)$ ist $x_{k+1} = x_k - \frac{1}{4} \cdot 2(x_k - 2) = x_k - \frac{1}{2}(x_k - 2)$, also $x_{k+1} - 2 = \frac{1}{2}(x_k - 2)$: Der Fehler halbiert sich. Mit $x_0 = 6$: $x_1 = 4$, $x_2 = 3$, $x_3 = \frac{5}{2}$, und $x_n - 2 = 4 \cdot 2^{-n}$. Aus $4 \cdot 2^{-n} \le \frac{1}{1000}$ folgt $2^n \ge 4000$, und $2^{11} = 2048 < 4000 \le 4096 = 2^{12}$: $n = 12$ Schritte.

(b) Allgemein ist $x_{k+1} - 2 = (1-2t)(x_k - 2)$, der Fehler wird mit $1-2t$ multipliziert. Das Verfahren konvergiert für jeden Startwert genau dann, wenn $|1-2t| < 1$ ist, also für $0 < t < 1$. Bei $t = 1$ ist der Faktor -1: $x_k = 6, -2, 6, -2, \ldots$ – das Verfahren springt zwischen zwei Punkten und konvergiert nicht. Bei $t = \frac{6}{5}$ ist der Faktor $-\frac{7}{5}$ mit Betrag > 1: $x_k = 6,\ -\frac{18}{5},\ \frac{246}{25},\ \ldots$ – der Fehler wächst.

(c) $\nabla f(x, y) = (x,\ 4y)$. Ein Schritt ist $(x - tx,\ y - 4ty) = \big((1-t)\,x,\ (1-4t)\,y\big)$. Mit $t = \frac{1}{5}$: $(x_1, y_1) = (\frac{16}{5}, \frac{1}{5})$, $(x_2, y_2) = (\frac{64}{25}, \frac{1}{25})$. Jede Koordinate wird mit ihrem eigenen Faktor multipliziert:

$$x_n = (1-t)^n\, x_0 = 4\,\big(\tfrac{4}{5}\big)^n, \qquad y_n = (1-4t)^n\, y_0 = \big(\tfrac{1}{5}\big)^n.$$

(d) Beide Faktoren müssen Betrag < 1 haben: $|1-t| < 1$ heißt $0 < t < 2$, $|1-4t| < 1$ heißt $0 < t < \frac{1}{2}$. Also konvergiert das Verfahren für jeden Startwert genau für $0 < t < \frac{1}{2}$. Die Zahl $\frac{1}{2} = \frac{2}{L}$ gehört zur größeren Krümmung $L = 4$; die steilere Richtung begrenzt die Schrittweite. Bei $t = \frac{1}{2}$ ist $y_n = (-1)^n\, y_0$: Die y-Koordinate springt, während $x_n = 4 \cdot 2^{-n}$ gegen 0 geht.

(e) $|1-t|$ fällt auf $(0, 1)$; $|1-4t|$ fällt bis $t = \frac{1}{4}$ und steigt danach als $4t - 1$. Das Maximum ist am kleinsten, wo $1 - t = 4t - 1$ ist, also bei $t = \frac{2}{5}$ mit $\rho = \frac{3}{5}$. (Für $t \le \frac{2}{5}$ ist $\rho \ge 1 - t \ge \frac{3}{5}$, für $t \ge \frac{2}{5}$ ist $\rho \ge 4t - 1 \ge \frac{3}{5}$.) Es ist $(\frac{3}{5})^n \le \frac{1}{1000} \iff n \ge \frac{\ln 1000}{\ln(5/3)} \approx 13{,}5$: 14 Schritte. Mit $t = \frac{1}{5}$ ist $\rho = \max\{\frac{4}{5}, \frac{1}{5}\} = \frac{4}{5}$ und $n \ge \frac{\ln 1000}{\ln(5/4)} \approx 31{,}0$: 31 Schritte. Eine Schrittweite, die beide Koordinaten in einem Schritt auf 0 bringt, bräuchte $t = 1$ und $t = \frac{1}{4}$ zugleich.

Aufgabe A4 (Lagrange)

(a) Minimieren Sie $\varphi(x, y) = x^2 + y^2$ unter der Nebenbedingung $f(x, y) = x + 2y = 5$: Prüfen Sie $\nabla f \neq 0$, stellen Sie $\nabla\varphi = \lambda\,\nabla f$ auf und bestimmen Sie λ und den Kandidaten p. Welchen Wert hat φ dort?

(b) Kontrollieren Sie das ohne Lagrange: Setzen Sie $x = 5 - 2y$ in φ ein.

(c) Bestimmen Sie die Lagrange-Kandidaten für $\varphi(x, y) = x+y$ unter $x^2+y^2 = 2$ und entscheiden Sie, wo das Maximum und wo das Minimum liegt.

Lösung. **(a)** $\nabla f = (1, 2) \neq 0$: die klassische Fassung ist anwendbar. $\nabla\varphi = (2x,\ 2y) = \lambda\,(1, 2)$ heißt $x = \frac{\lambda}{2}$ und $y = \lambda$. Eingesetzt in $x + 2y = 5$: $\frac{\lambda}{2} + 2\lambda = 5$, also $\lambda = 2$ und $p = (1, 2)$ mit $\varphi(p) = 1 + 4 = 5$. Geometrisch ist p der Punkt der Geraden, der dem Nullpunkt am nächsten liegt, im Abstand $\sqrt{5}$; $\nabla\varphi(p) = (2, 4) = 2\,(1, 2)$ steht senkrecht auf der Geraden.

(b) Mit $x = 5 - 2y$ ist $\varphi = (5 - 2y)^2 + y^2 = 5y^2 - 20y + 25 = 5\,(y - 2)^2 + 5 \geq 5$, mit Gleichheit genau für $y = 2$, also $x = 1$. Der Kandidat ist das Minimum und der einzige Minimalpunkt.

(c) $\nabla f = (2x, 2y) \neq 0$ auf der Kreislinie. Aus $(1, 1) = \lambda\,(2x,\ 2y)$ folgt $x = y = \frac{1}{2\lambda}$, und $x^2 + y^2 = 2$ gibt $2x^2 = 2$, also $x = \pm 1$. Kandidaten: $(1, 1)$ mit $\lambda = \frac{1}{2}$ und $\varphi = 2$, und $(-1, -1)$ mit $\lambda = -\frac{1}{2}$ und $\varphi = -2$. Die Kreislinie ist beschränkt und abgeschlossen, also werden Maximum und Minimum angenommen und liegen unter den Kandidaten: Das Maximum 2 liegt in $(1, 1)$, das Minimum -2 in $(-1, -1)$. Lagrange liefert die Kandidaten, der Vergleich der Werte entscheidet.

Teil B: In Lean

Aufgabe B1 (Hoch-, Tief- und Sattelpunkte: zu A2 (b) und (c))

Punkte der Ebene sind in diesem Teil Paare `p` mit den Koordinaten `p.1` und `p.2`; `landschaft` ist die Funktion f aus A2. `IsLocalMin f p` heißt: Schließlich längs $\mathcal{N}(p)$ ist $f(p) \le f(q)$; `IsLocalMax f p` ebenso mit $f(p) \ge f(q)$. Auf Paaren rechnet Mathlib mit der Maximumsnorm: Die Kugel vom Radius δ um p ist das Quadrat aus allen q mit $|q_1 - p_1| < \delta$ und $|q_2 - p_2| < \delta$.

(a) Für $x, y \ge -2$ ist $f(1,1) \le f(x,y)$.

(b) f hat in $(1,1)$ ein lokales Minimum.

(c) f hat in $(1,-1)$ weder ein lokales Minimum noch ein lokales Maximum.

Hinweis: (a) ist die Rechnung aus A2 (b) mit $x = 1+h$ und $y = 1+k$: $f(x,y) - f(1,1) = (x-1)^2(x+2) + (y-1)^2(y+2)$. Diese Gleichung zeigt `ring`, die Vorzeichen der Summanden `mul_nonneg` und `sq_nonneg`. (b) `filter_upwards` mit der Kugel vom Radius 3 um $(1,1)$; `Prod.dist_eq`, `max_lt_iff`, `Real.dist_eq` und `abs_lt` machen aus „p liegt in der Kugel“ Ungleichungen für die Koordinaten. (c) geht wie B3 (b) auf Blatt 5: `Metric.eventually_nhds_iff` liefert eine Kugel vom Radius δ; die Punkte $(1, -1+t)$ und $(1+t, -1)$ mit $t = \min(\delta/2, 1)$ liegen darin. Ungleichungen mit Produkten erledigt `nlinarith`, wenn man ihm $t^2(3-t) > 0$ bzw. $t^2(3+t) > 0$ als `mul_pos …` mitgibt.

Lean `lean/Blatt06.lean`

```
/-- Die Funktion aus A2, `f(x, y) = x^3 - 3x + y^3 - 3y`. Punkte der Ebene sind
    Paare `p` mit den Koordinaten `p.1` und `p.2`. -/
def landschaft (p : ℝ × ℝ) : ℝ := p.1 ^ 3 - 3 * p.1 + (p.2 ^ 3 - 3 * p.2)

-- `IsLocalMin f p` heisst: schliesslich laengs `𝒩 p` ist `f p ≤ f q`;
-- `IsLocalMax f p` ebenso mit `f q ≤ f p`. Auf Paaren rechnet Mathlib mit
-- der Maximumsnorm: Die Kugel vom Radius δ um p ist ein Quadrat.

/-- Aufgabe B1 (a) (A2 (b)): Fuer `x, y ≥ -2` ist `f(1, 1) ≤ f(x, y)`. -/
theorem landschaft_abschaetzung (p : ℝ × ℝ) (hx : -2 ≤ p.1)
    (hy : -2 ≤ p.2) : landschaft (1, 1) ≤ landschaft p := by
  -- Nuetzlich: die Rechnung von Papier als Gleichung `have e : ... := by
  -- simp only [landschaft]; ring`, dann mul_nonneg, sq_nonneg, linarith
  sorry

/-- Aufgabe B1 (b) (A2 (b)): `f` hat in `(1, 1)` ein lokales Minimum. -/
theorem landschaft_tal : IsLocalMin landschaft (1, 1) := by
  -- Nuetzlich: filter_upwards mit Metric.ball_mem_nhds (die Kugel vom
  -- Radius 3 um (1, 1)), Metric.mem_ball, Prod.dist_eq, max_lt_iff,
  -- Real.dist_eq, abs_lt, linarith; dann (a)
  sorry

/-- Aufgabe B1 (c) (A2 (c)): In `(1, -1)` hat `f` weder ein lokales Minimum
    noch ein lokales Maximum: ein Sattelpunkt. -/
theorem landschaft_sattel :
    ¬ IsLocalMin landschaft (1, -1) ∧ ¬ IsLocalMax landschaft (1, -1) := by
  -- Nuetzlich: wie betrag_nicht_klein_o auf Blatt 5 (B3 (b)):
   Metric.eventually_nhds_iff
  -- liefert ein δ > 0. Die Punkte (1, -1 + t) und (1 + t, -1) mit
  -- t = min (δ / 2) 1 liegen in der Kugel vom Radius δ (lt_min, half_pos,
  -- min_le_left, min_le_right, half_lt_self, Prod.dist_eq, max_lt_iff,
  -- Real.dist_eq, abs_of_pos). Dann simp only [landschaft] und nlinarith mit
  -- dem Hinweis mul_pos (mul_pos ht ht) _ (mit ht : 0 < t) fuer
  -- t^2 (3 - t) > 0 bzw. t^2 (3 + t) > 0
  sorry
```

Lösung. (a) Die Gleichung `e` ist die Rechnung von Papier; nach dem Einsetzen der Definition rechnet `ring` sie nach. Beide Summanden sind ein Quadrat mal einer Zahl ≥ 0, also ≥ 0, und `linarith` schließt. (b) Nach `filter_upwards` ist für einen Punkt p der Kugel vom Radius 3 um $(1,1)$ zu zeigen, dass $f(1,1) \leq f(p)$ ist. In der Maximumsnorm heißt „p liegt in der Kugel“: $|p_1 - 1| < 3$ und $|p_2 - 1| < 3$. Also ist $p_1, p_2 > -2$, und (a) greift. (c) Angenommen, $(1,-1)$ ist ein lokales Minimum. Dann ist $f(1,-1) \leq f(q)$ auf einer Kugel vom Radius δ. Der Punkt $(1,-1+t)$ liegt darin, und dort ist $f = t^2\,(t-3) < 0 = f(1,-1)$ – die zweite Schnittkurve aus A2 (c); das $\min(\cdot,1)$ sorgt für $t < 3$. Ebenso widerlegt $(1+t,-1)$ mit $f = t^2\,(3+t) > 0$ ein lokales Maximum. Das lokale Maximum in $(-1,-1)$ aus A2 (b) ist nicht formalisiert; es geht wie (a) und (b), mit $4 - f(x,y) = (x+1)^2\,(2-x) + (y+1)^2\,(2-y)$.

Lean, Lösung `loesungen/Blatt06_Loesung.lean`

```
/-- Die Funktion aus A2, `f(x, y) = x^3 - 3x + y^3 - 3y`. Punkte der Ebene sind
    Paare `p` mit den Koordinaten `p.1` und `p.2`. -/
def landschaft (p : ℝ × ℝ) : ℝ := p.1 ^ 3 - 3 * p.1 + (p.2 ^ 3 - 3 * p.2)

-- `IsLocalMin f p` heisst: schliesslich laengs `𝓝 p` ist `f p ≤ f q`;
-- `IsLocalMax f p` ebenso mit `f q ≤ f p`. Auf Paaren rechnet Mathlib mit
-- der Maximumsnorm: Die Kugel vom Radius δ um p ist ein Quadrat.

/-- Aufgabe B1 (a) (A2 (b)): Fuer `x, y ≥ -2` ist `f(1, 1) ≤ f(x, y)`. -/
theorem landschaft_abschaetzung (p : ℝ × ℝ) (hx : -2 ≤ p.1)
    (hy : -2 ≤ p.2) : landschaft (1, 1) ≤ landschaft p := by
  -- Nuetzlich: die Rechnung von Papier als Gleichung `have e : ... := by
  -- simp only [landschaft]; ring`, dann mul_nonneg, sq_nonneg, linarith
  -- A2 (b) mit x = 1 + h und y = 1 + k:
  -- f(x, y) - f(1, 1) = (x - 1)^2 (x + 2) + (y - 1)^2 (y + 2)
  have e : landschaft p - landschaft (1, 1) =
      (p.1 - 1) ^ 2 * (p.1 + 2) + (p.2 - 1) ^ 2 * (p.2 + 2) := by
    simp only [landschaft]
    ring
  -- Quadrat mal Zahl ≥ 0 ist ≥ 0, fuer beide Summanden
  have h1 : 0 ≤ (p.1 - 1) ^ 2 * (p.1 + 2) :=
    mul_nonneg (sq_nonneg _) (by linarith)
  have h2 : 0 ≤ (p.2 - 1) ^ 2 * (p.2 + 2) :=
    mul_nonneg (sq_nonneg _) (by linarith)
  linarith

/-- Aufgabe B1 (b) (A2 (b)): `f` hat in `(1, 1)` ein lokales Minimum. -/
theorem landschaft_tal : IsLocalMin landschaft (1, 1) := by
  -- Nuetzlich: filter_upwards mit Metric.ball_mem_nhds (die Kugel vom
  -- Radius 3 um (1, 1)), Metric.mem_ball, Prod.dist_eq, max_lt_iff,
  -- Real.dist_eq, abs_lt, linarith; dann (a)
  -- Die Kugel vom Radius 3 um (1, 1) ist gross.
  filter_upwards [Metric.ball_mem_nhds ((1, 1) : ℝ × ℝ) three_pos] with p hp
  -- In ihr ist |x - 1| < 3 und |y - 1| < 3, also x > -2 und y > -2.
  rw [Metric.mem_ball, Prod.dist_eq, max_lt_iff, Real.dist_eq, Real.dist_eq,
    abs_lt, abs_lt] at hp
  apply landschaft_abschaetzung p
  · linarith [hp.1.1]
  · linarith [hp.2.1]

/-- Aufgabe B1 (c) (A2 (c)): In `(1, -1)` hat `f` weder ein lokales Minimum
    noch ein lokales Maximum: ein Sattelpunkt. -/
theorem landschaft_sattel :
    ¬ IsLocalMin landschaft (1, -1) ∧ ¬ IsLocalMax landschaft (1, -1) := by
  -- Nuetzlich: wie betrag_nicht_klein_o auf Blatt 5 (B3 (b)):
  Metric.eventually_nhds_iff
```

```
-- liefert ein δ > 0. Die Punkte (1, -1 + t) und (1 + t, -1) mit
-- t = min (δ / 2) 1 liegen in der Kugel vom Radius δ (lt_min, half_pos,
-- min_le_left, min_le_right, half_lt_self, Prod.dist_eq, max_lt_iff,
-- Real.dist_eq, abs_of_pos). Dann simp only [landschaft] und nlinarith mit
-- dem Hinweis mul_pos (mul_pos ht ht) _ (mit ht : 0 < t) fuer
-- t^2(3 - t) > 0 bzw. t^2(3 + t) > 0
constructor
· -- Kein Minimum: in y-Richtung faellt f, f(1, -1 + t) = t^2(t - 3) < 0.
  intro h
  -- Auf einer Kugel vom Radius δ ist f(1, -1) ≤ f(q).
  obtain ⟨δ, hδ, hball⟩ := Metric.eventually_nhds_iff.1 h
  set t := min (δ / 2) 1
  have ht : 0 < t := lt_min (half_pos hδ) one_pos
  have ht1 : t ≤ 1 := min_le_right _ _                -- also t < 3
  have htδ : t < δ := lt_of_le_of_lt (min_le_left _ _) (half_lt_self hδ)
  -- Der Punkt (1, -1 + t) liegt in der Kugel.
  have h3 := hball (y := (1, -1 + t)) (by
    rw [Prod.dist_eq, max_lt_iff, Real.dist_eq, Real.dist_eq]
    constructor
    · simpa using hδ
    · rw [show -1 + t - -1 = t by ring, abs_of_pos ht]
      exact htδ)
  simp only [landschaft] at h3          -- h3 : f(1, -1) ≤ f(1, -1 + t)
  nlinarith [mul_pos (mul_pos ht ht) (by linarith : (0 : ℝ) < 3 - t)]
· -- Kein Maximum: in x-Richtung steigt f, f(1 + t, -1) = t^2(3 + t) > 0.
  intro h
  obtain ⟨δ, hδ, hball⟩ := Metric.eventually_nhds_iff.1 h
  set t := min (δ / 2) 1
  have ht : 0 < t := lt_min (half_pos hδ) one_pos
  have htδ : t < δ := lt_of_le_of_lt (min_le_left _ _) (half_lt_self hδ)
  have h3 := hball (y := (1 + t, -1)) (by
    rw [Prod.dist_eq, max_lt_iff, Real.dist_eq, Real.dist_eq]
    constructor
    · rw [show 1 + t - 1 = t by ring, abs_of_pos ht]
      exact htδ
    · simpa using hδ)
  simp only [landschaft] at h3          -- h3 : f(1 + t, -1) ≤ f(1, -1)
  nlinarith [mul_pos (mul_pos ht ht) (by linarith : (0 : ℝ) < 3 + t)]
```

Aufgabe B2 (Gradientenabstieg: zu A3 (c) und (d))

Punkte der Ebene sind in diesem Teil Paare `p` mit den Koordinaten `p.1` und `p.2`. Auf Paaren rechnet Mathlib mit der Maximumsnorm statt der euklidischen Norm; für die Konvergenz macht das keinen Unterschied – beide Male heißt sie: Konvergenz beider Koordinaten. Vorgegeben sind der Gradient von $f(x,y) = \frac{1}{2}(ax^2 + by^2)$ als `gradF` (A3 hat $a = 1$, $b = 4$; dass er der Gradient von f ist, beweisen wir in Lean nicht), ein Schritt `gdSchritt2` und die Iteration `gdIter2`, gebaut wie `gdSchritt` und `gdIter` in der Vorlesung. Vorgegeben ist auch das Lemma `gd2_schritt`: Ein Schritt multipliziert die Koordinaten mit $1 - ta$ bzw. $1 - tb$.

(a) Die geschlossene Form aus A3 (c): $x_n = (1-ta)^n\, x_0$ und $y_n = (1-tb)^n\, y_0$.

(b) $|1 - ta| < 1$ und $|1 - tb| < 1$ gelten genau dann, wenn $0 < t < 2/b$ – für $0 < a \le b$.

(c) Für $0 < t < 2/b$ konvergiert das Verfahren von jedem Startwert aus gegen $(0,0)$ – die Richtung „⇐“ von A3 (d).

Hinweis: (a) geht wie `gd_q_fehler` aus der Vorlesung; nach `ext` ist für jede Koordinate eine Gleichung zwischen reellen Zahlen übrig. (b) geht wie `schrittweite` aus der Vorlesung. In (c) liefert (b) die beiden Beträge < 1; dann weiter wie in `gd_q_tendsto`. `Tendsto.prodMk_nhds` setzt die Grenzwerte der beiden Koordinaten zu einem Grenzwert in der Ebene zusammen.

Lean `lean/Blatt06.lean`

```
/-- Der Gradient von `f(x, y) = (a x^2 + b y^2)/2` -- fuer `a = 1`, `b = 4` die
    Funktion aus A3. Punkte der Ebene sind Paare `p` mit den Koordinaten `p.1`
    und `p.2`. -/
def gradF (a b : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := (a * p.1, b * p.2)

/-- Ein Schritt des Gradientenabstiegs, `p - t ∇f(p)` (wie gdSchritt). -/
def gdSchritt2 (a b t : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := p - t • gradF a b p

/-- Die Iteration `p₀, p₁, p₂, ...` (wie gdIter in der Vorlesung). -/
def gdIter2 (a b t : ℝ) (p₀ : ℝ × ℝ) : ℕ → ℝ × ℝ
  | 0 => p₀
  | n + 1 => gdSchritt2 a b t (gdIter2 a b t p₀ n)

/-- Vorgegeben: Ein Schritt multipliziert die Koordinaten mit `1 - t a` bzw.
    `1 - t b`. `ext` zerlegt eine Gleichung zwischen Paaren in zwei
    Gleichungen, eine fuer jede Koordinate. -/
theorem gd2_schritt (a b t : ℝ) (p : ℝ × ℝ) :
    gdSchritt2 a b t p = ((1 - t * a) * p.1, (1 - t * b) * p.2) := by
  ext
  · simp [gdSchritt2, gradF]
    ring
  · simp [gdSchritt2, gradF]
    ring

/-- Aufgabe B2 (a) (A3 (c)): die geschlossene Form. -/
theorem gd2_formel (a b t : ℝ) (p₀ : ℝ × ℝ) (n : ℕ) :
    gdIter2 a b t p₀ n = ((1 - t * a) ^ n * p₀.1, (1 - t * b) ^ n * p₀.2) := by
  -- Nuetzlich: wie gd_q_fehler in der Vorlesung, mit gd2_schritt; fuer
  -- Gleichungen zwischen Paaren hilft ext (siehe gd2_schritt)
  sorry

/-- Aufgabe B2 (b) (A3 (d), Rechnung): Beide Faktoren haben genau dann Betrag
    `< 1`, wenn `0 < t < 2/b`. -/
theorem schrittweite2 (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) :
    (|1 - t * a| < 1 ∧ |1 - t * b| < 1) ↔ (0 < t ∧ t < 2 / b) := by
```

```
  -- Nuetzlich: wie schrittweite in der Vorlesung: abs_lt, lt_div_iff₀
  -- (dafuer erst 0 < b zeigen), constructor, rintro, nlinarith
  sorry

/-- Aufgabe B2 (c) (A3 (d), eine Richtung): Fuer `0 < t < 2/b` konvergiert das
    Verfahren von jedem Startwert aus gegen das Minimum `(0, 0)`. -/
theorem gd2_konvergenz (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) (ht : 0 < t)
    (ht' : t < 2 / b) (p₀ : ℝ × ℝ) :
    Tendsto (gdIter2 a b t p₀) atTop (𝓝 (0, 0)) := by
  -- Nuetzlich: schrittweite2; aus gd_q_tendsto in der Vorlesung
  -- tendsto_pow_atTop_nhds_zero_of_norm_lt_one, Real.norm_eq_abs und
  -- Tendsto.mul_const; Tendsto.prodMk_nhds; funext (gd2_formel a b t p₀)
  -- ist die Formel als Gleichung zwischen Folgen
  sorry
```

Lösung. (a) Induktion über n: Der Fall 0 ist eine Rechnung mit `simp`. Im Schritt entfaltet `simp only` eine Stufe von `gdIter2`, dann setzt `rw` den Schritt `gd2_schritt` und die Induktionsannahme `ih` ein. Nach `ext` rechnet `simp only` die Koordinaten der Paare aus, und `ring` fasst $(1-ta)\,(1-ta)^k$ zu $(1-ta)^{k+1}$ zusammen. (b) `abs_lt` zerlegt die beiden Beträge, `lt_div_iff₀` macht aus $t < 2/b$ die Ungleichung $t\,b < 2$ (dafür braucht man $0 < b$, das `linarith` aus $0 < a \le b$ schließt). In beiden Richtungen bleiben Ungleichungen mit Produkten wie $t\,a$, die `nlinarith` erledigt – für $t\,a \le t\,b$ multipliziert es $t > 0$ mit $a \le b$. (c) Wie in der Vorlesung gehen $(1-ta)^n$ und $(1-tb)^n$ gegen 0, mal der Koordinate des Startwerts ebenfalls. Mit `funext` und (a) wird die Iteration durch die Formel ersetzt, und `Tendsto.prodMk_nhds` setzt die beiden Koordinaten zusammen. Die Richtung „⇒“ von A3 (d) – für $t \ge 2/b$ divergiert das Verfahren mit dem Startwert $(0,1)$ – ist nicht formalisiert.

Lean, Lösung loesungen/Blatt06_Loesung.lean

```
/-- Der Gradient von `f(x, y) = (a x^2 + b y^2)/2` -- fuer `a = 1`, `b = 4` die
    Funktion aus A3. Punkte der Ebene sind Paare `p` mit den Koordinaten `p.1`
    und `p.2`. -/
def gradF (a b : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := (a * p.1, b * p.2)

/-- Ein Schritt des Gradientenabstiegs, `p - t ∇f(p)` (wie gdSchritt). -/
def gdSchritt2 (a b t : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := p - t • gradF a b p

/-- Die Iteration `p₀, p₁, p₂, ...` (wie gdIter in der Vorlesung). -/
def gdIter2 (a b t : ℝ) (p₀ : ℝ × ℝ) : ℕ → ℝ × ℝ
  | 0 => p₀
  | n + 1 => gdSchritt2 a b t (gdIter2 a b t p₀ n)

/-- Vorgegeben: Ein Schritt multipliziert die Koordinaten mit `1 - t a` bzw.
    `1 - t b`. `ext` zerlegt eine Gleichung zwischen Paaren in zwei
    Gleichungen, eine fuer jede Koordinate. -/
theorem gd2_schritt (a b t : ℝ) (p : ℝ × ℝ) :
    gdSchritt2 a b t p = ((1 - t * a) * p.1, (1 - t * b) * p.2) := by
  ext
  · simp [gdSchritt2, gradF]
    ring
  · simp [gdSchritt2, gradF]
    ring

/-- Aufgabe B2 (a) (A3 (c)): die geschlossene Form. -/
theorem gd2_formel (a b t : ℝ) (p₀ : ℝ × ℝ) (n : ℕ) :
    gdIter2 a b t p₀ n = ((1 - t * a) ^ n * p₀.1, (1 - t * b) ^ n * p₀.2) := by
  -- Nuetzlich: wie gd_q_fehler in der Vorlesung, mit gd2_schritt; fuer
  -- Gleichungen zwischen Paaren hilft ext (siehe gd2_schritt)
```

```
  induction n with
  | zero => simp [gdIter2]
  | succ k ih =>
      simp only [gdIter2]
      rw [gd2_schritt, ih]
      ext
      · simp only
        ring
      · simp only
        ring

/-- Aufgabe B2 (b) (A3 (d), Rechnung): Beide Faktoren haben genau dann Betrag
    '< 1', wenn '0 < t < 2/b'. -/
theorem schrittweite2 (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) :
    (|1 - t * a| < 1 ∧ |1 - t * b| < 1) ↔ (0 < t ∧ t < 2 / b) := by
  -- Nuetzlich: wie schrittweite in der Vorlesung: abs_lt, lt_div_iff₀
  -- (dafuer erst 0 < b zeigen), constructor, rintro, nlinarith
  have hb : 0 < b := by linarith
  rw [abs_lt, abs_lt, lt_div_iff₀ hb]
  constructor
  · rintro ⟨⟨h1, h2⟩, h3, h4⟩
    exact ⟨by nlinarith, by nlinarith⟩
  · rintro ⟨h1, h2⟩
    exact ⟨⟨by nlinarith, by nlinarith⟩, by nlinarith, by nlinarith⟩

/-- Aufgabe B2 (c) (A3 (d), eine Richtung): Fuer '0 < t < 2/b' konvergiert das
    Verfahren von jedem Startwert aus gegen das Minimum '(0, 0)'. -/
theorem gd2_konvergenz (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) (ht : 0 < t)
    (ht' : t < 2 / b) (p₀ : ℝ × ℝ) :
    Tendsto (gdIter2 a b t p₀) atTop (𝓝 (0, 0)) := by
  -- Nuetzlich: schrittweite2; aus gd_q_tendsto in der Vorlesung
  -- tendsto_pow_atTop_nhds_zero_of_norm_lt_one, Real.norm_eq_abs und
  -- Tendsto.mul_const; Tendsto.prodMk_nhds; funext (gd2_formel a b t p₀)
  -- ist die Formel als Gleichung zwischen Folgen
  obtain ⟨hA, hB⟩ := (schrittweite2 a b t ha hab).mpr ⟨ht, ht'⟩
  have hpa : Tendsto (fun n : ℕ => (1 - t * a) ^ n) atTop (𝓝 0) :=
    tendsto_pow_atTop_nhds_zero_of_norm_lt_one (by rwa [Real.norm_eq_abs])
  have hpb : Tendsto (fun n : ℕ => (1 - t * b) ^ n) atTop (𝓝 0) :=
    tendsto_pow_atTop_nhds_zero_of_norm_lt_one (by rwa [Real.norm_eq_abs])
  have hx : Tendsto (fun n : ℕ => (1 - t * a) ^ n * p₀.1) atTop (𝓝 0) := by
    simpa using hpa.mul_const p₀.1
  have hy : Tendsto (fun n : ℕ => (1 - t * b) ^ n * p₀.2) atTop (𝓝 0) := by
    simpa using hpb.mul_const p₀.2
  have hf : gdIter2 a b t p₀ =
      fun n => ((1 - t * a) ^ n * p₀.1, (1 - t * b) ^ n * p₀.2) :=
    funext (gd2_formel a b t p₀)
  rw [hf]
  exact hx.prodMk_nhds hy
```

Aufgabe B3 (Lagrange: zu A4 (a) und (b))

Lagrange liefert den Kandidaten; dass er wirklich das Optimum ist, bestätigt eine elementare Ungleichung, wie bei $xy \le \frac{1}{4}$ unter $x + y = 1$ in der Vorlesung.

(a) Auf der Geraden $x + 2y = 5$ ist $x^2 + y^2 \ge 5$.

(b) Der Wert 5 wird nur im Lagrange-Punkt $(1, 2)$ angenommen.

Hinweis: Es ist $5\,(x^2+y^2) = (x+2y)^2 + (2x-y)^2$. Geben Sie `nlinarith` den Hinweis `sq_nonneg (2 * x - y)`. In (b) ist dann $(2x-y)^2 = 0$, und aus `pow_eq_zero_iff` folgt $2x - y = 0$.

Lean `lean/Blatt06.lean`

```
/-- Aufgabe B3 (a) (A4 (a)): Auf der Geraden ‘x + 2y = 5‘ ist ‘x² + y² ≥ 5‘;
    der Lagrange-Kandidat ‘(1, 2)‘ liefert den Wert ‘5‘. -/
theorem abstand_gerade (x y : ℝ) (h : x + 2 * y = 5) : 5 ≤ x ^ 2 + y ^ 2 := by
  -- Nuetzlich: wie xy_le_quarter in der Vorlesung: nlinarith mit dem Hinweis
  -- sq_nonneg (2 * x - y); denn 5·(x² + y²) = (x + 2y)² + (2x - y)²
  sorry

/-- Aufgabe B3 (b) (A4 (a)): Der Wert ‘5‘ wird nur in ‘(1, 2)‘ angenommen. -/
theorem abstand_gerade_eindeutig (x y : ℝ) (h : x + 2 * y = 5)
    (hmin : x ^ 2 + y ^ 2 = 5) : x = 1 ∧ y = 2 := by
  -- Nuetzlich: Aus der Gleichung 5·(x² + y²) = (x + 2y)² + (2x - y)² folgt
  -- (2x - y)² = 0 (nlinarith), also 2x - y = 0 (pow_eq_zero_iff); dann linarith
  sorry
```

Lösung. (a) Die Identität $5\,(x^2+y^2) = (x+2y)^2 + (2x-y)^2$ ist die Cauchy-Schwarz-Ungleichung für $(1,2)$ und (x,y). Mit $x+2y=5$ steht $5\,(x^2+y^2) = 25 + (2x-y)^2 \geq 25$; `nlinarith` multipliziert dafür die Annahme mit sich selbst und nimmt $(2x-y)^2 \geq 0$ als Hinweis dazu. (b) Aus $x^2+y^2=5$ folgt in derselben Identität $(2x-y)^2 = 0$, also $y = 2x$; mit $x+2y=5$ ist $5x=5$, also $x=1$ und $y=2$ – der Kandidat aus A4 (a), und er ist der einzige.

Lean, Lösung `loesungen/Blatt06_Loesung.lean`

```
/-- Aufgabe B3 (a) (A4 (a)): Auf der Geraden ‘x + 2y = 5‘ ist ‘x² + y² ≥ 5‘;
    der Lagrange-Kandidat ‘(1, 2)‘ liefert den Wert ‘5‘. -/
theorem abstand_gerade (x y : ℝ) (h : x + 2 * y = 5) : 5 ≤ x ^ 2 + y ^ 2 := by
  -- Nuetzlich: wie xy_le_quarter in der Vorlesung: nlinarith mit dem Hinweis
  -- sq_nonneg (2 * x - y); denn 5·(x² + y²) = (x + 2y)² + (2x - y)²
  nlinarith [sq_nonneg (2 * x - y)]

/-- Aufgabe B3 (b) (A4 (a)): Der Wert ‘5‘ wird nur in ‘(1, 2)‘ angenommen. -/
theorem abstand_gerade_eindeutig (x y : ℝ) (h : x + 2 * y = 5)
    (hmin : x ^ 2 + y ^ 2 = 5) : x = 1 ∧ y = 2 := by
  -- Nuetzlich: Aus der Gleichung 5·(x² + y²) = (x + 2y)² + (2x - y)² folgt
  -- (2x - y)² = 0 (nlinarith), also 2x - y = 0 (pow_eq_zero_iff); dann linarith
  have h1 : (2 * x - y) ^ 2 = 0 := by nlinarith
  have h2 : 2 * x - y = 0 := pow_eq_zero_iff (by norm_num) |>.1 h1
  constructor <;> linarith
```